

Automated Bidding Model for Used Car Auctions

Coding Club Recruitment — Task 1 Technical Report
Om Sharma

Competition	Coding Club Recruitment — Task 1
Model	XGBoost Regressor (Tuned, RandomizedSearchCV)
Features	15 total: 7 numeric (4 engineered) + 8 label-encoded
Target Transform	$\log_{1p}(\text{sellingprice}) \rightarrow \text{expm1}$ at inference
Val RMSE	\$1,620 (14.3% improvement over default \$1,890)
Starting Bankroll	\$500,000
Bid Strategy	Sigmoid-proportional deterministic sequence
Profit Rule	Win = positive profit only; folding is free

§1 Data Cleaning and Wrangling

1.1 Dataset Overview and Train-Validation Split

The raw dataset contained **447,048 rows** across 12 columns. Before any cleaning, an 80/20 stratified train-validation split (`random_state=42`) was performed. Every imputation statistic, encoder, and lookup table was computed on the training partition only and applied identically to validation — ensuring zero data leakage.

1.2 Missing Value Analysis

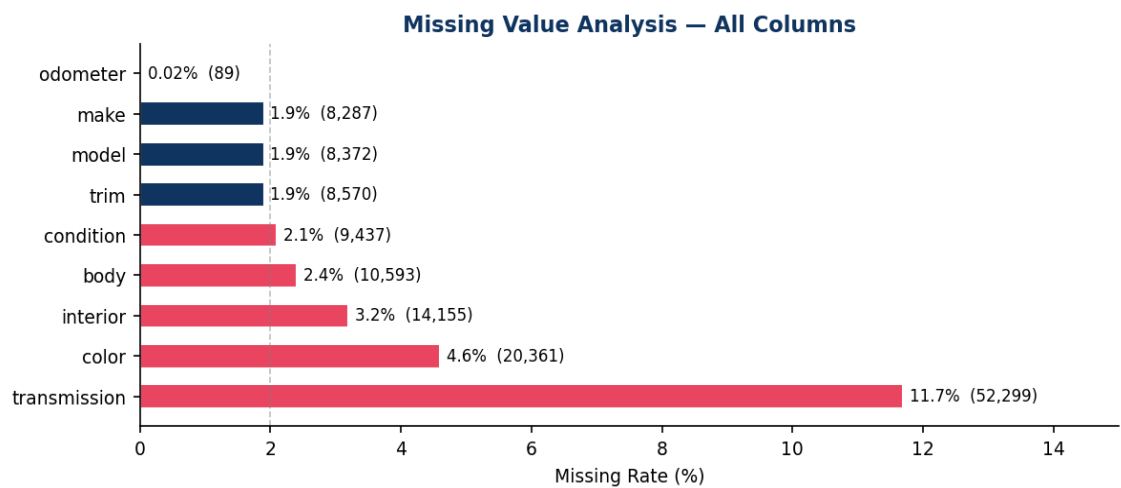


Figure 1: Missing rates across all columns. transmission (11.7%) and color (4.6%) are highest; year and sellingprice have zero missing values.

A structural dependency was identified: whenever **make** was missing, **model** and **trim** were also missing in 100% of cases. Since these three columns are dominant price drivers, imputing them would introduce severe false signal. The 8,287 affected rows (<2%) were dropped — the only rows removed from the entire dataset.

1.3 Hierarchical Imputation Strategy

Each column was classified by missing-data type (MCAR/MAR/MNAR) and imputed accordingly:

Column	Missing %	Type	Method	Justification
condition	2.1%	MAR	Median	Right-skewed; median robust to outliers
odometer	0.02%	MAR	Median	Extreme right skew; group median by year+model
transmission	11.7%	MAR	Mode (make+model → make → color)	Body-type dependency confirmed by grouped analysis
color	4.6%	MCAR	Mode	Most frequent = 'white'; cosmetic, low signal
interior	3.2%	MCAR	Mode	Most frequent = 'black'
body	2.4%	MAR	'unknown' token	High cardinality; mode would mislead
trim	1.9%	MAR	'unknown' token	4,500+ unique values
make/model/state	1.9%	MNAR	Row dropped	Structurally critical; imputing fabricates identity

1.4 Outlier Detection — Log-IQR Method

Standard IQR on raw **sellingprice** (skewness > 2.1, excess kurtosis > 15) produced a nonsensical upper bound of \$35,250 and a negative lower bound — a direct consequence of applying a symmetric rule to a right-skewed distribution. The fix: apply IQR on the log-transformed distribution:

```
upper_bound = exp( Q3(ln p) + 1.5 * IQR(ln p) ) - 1
```

This yielded bounds of **\$1,655 – \$77,347** for price and **\$4,345 – \$633,821** for odometer. High-price outliers (>\$77k) were confirmed genuine (Mercedes-Benz, Porsche, Ferrari; mean condition 4.45) and retained entirely. Odometer values <10 or >900,000 were flagged as data-entry errors, set to NaN, and re-imputed through the hierarchical pipeline.

§2 Exploratory Data Analysis

2.1 Correlation Analysis — Spearman vs Pearson

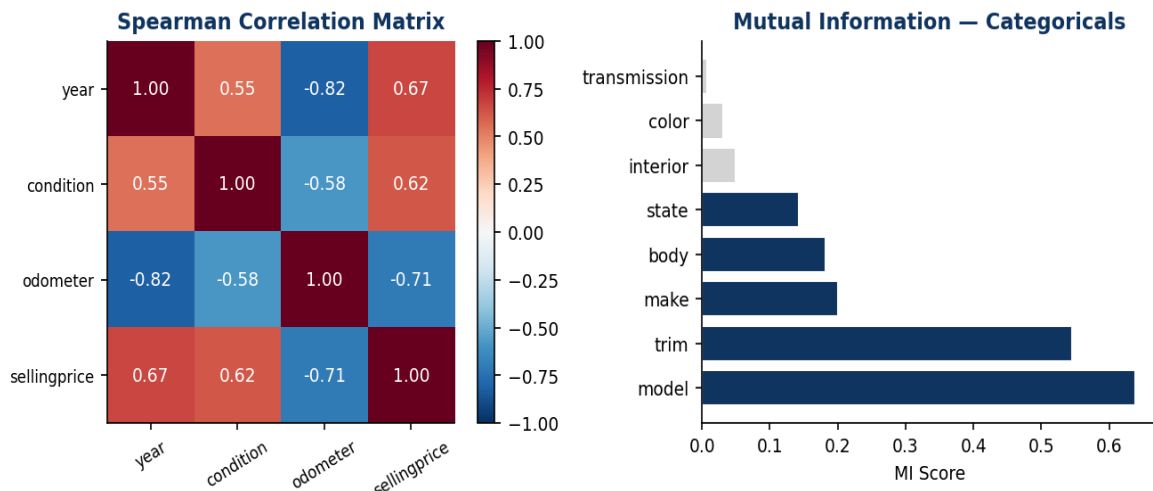


Figure 2: (Left) Spearman rank correlation matrix. Spearman used over Pearson due to non-normal, right-skewed distributions. (Right) Mutual Information scores for categorical features — model and trim dominate.

Spearman (rank-based) was chosen over Pearson because both **odometer** and **sellingprice** are right-skewed — Pearson's assumption of linearity and normality is violated. Key findings: odometer→price (**-0.71**, strongest numeric predictor), year→price (**+0.67**), condition→price (**+0.62**), and year↔odometer (**-0.82**, severe multicollinearity). MI scores confirmed **model (0.639)** and **trim (0.545)** are the most informative categorical — far more than make (0.201) alone. transmission carries near-zero marginal signal (0.008) yet was retained as it shows conditional effects when interacted with body type.

2.2 Depreciation Across Body Types

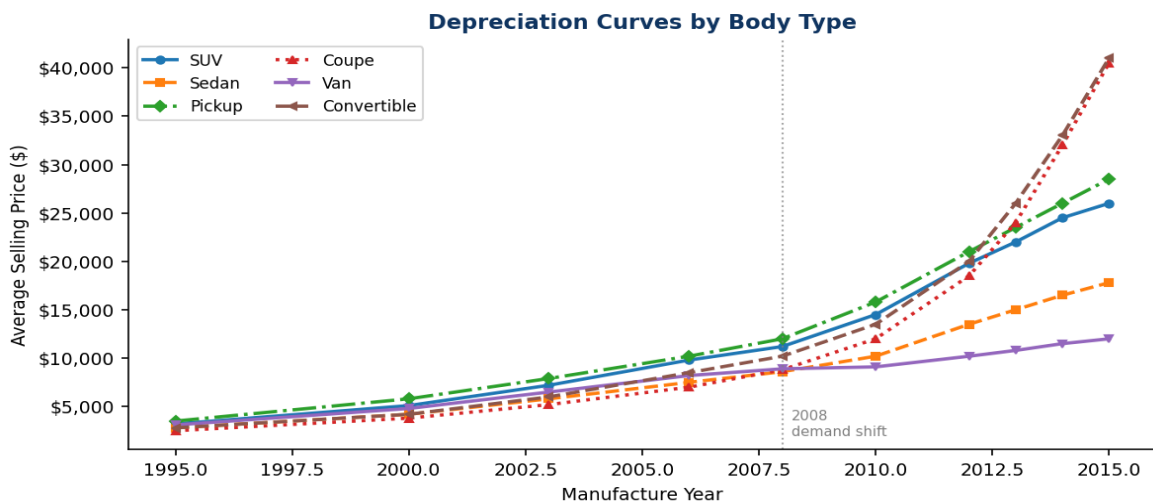


Figure 3: Depreciation curves by body type. Coupe/convertible diverge post-2010 due to composition effect. Van separates from SUV after 2008 demand shift. Pickup retains value best.

Three non-obvious patterns emerged: (1) **Composition effect** — coupes reach \$40,500 by 2015 not because coupes appreciated, but because the mix shifted from economy coupes (pre-2005) to luxury/sports vehicles (BMW 4-series, Porsche 911) post-2010. (2) **Pre-2000 convergence** — all body types collapse within \$500–2,000 before 1999; body type loses virtually all predictive power for vehicles older than 15–20 years. (3) **Demand-side van depreciation** — vans tracked SUVs until 2008, then diverged by \$10,000 as consumer preference shifted to

crossovers.

2.3 Condition Non-Linearity — Structural Break at 3.0

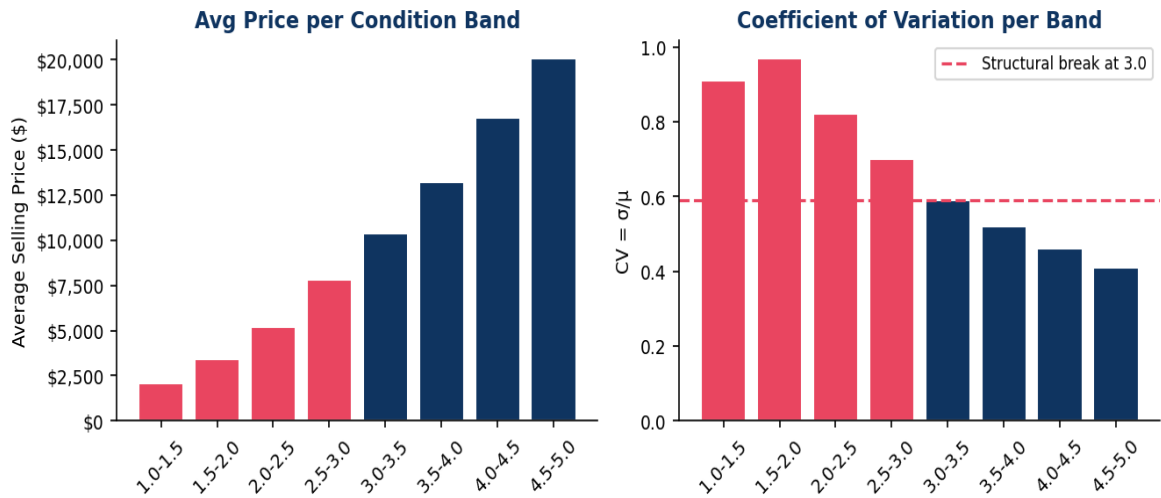


Figure 4: (Left) Average price per condition band — monotonically increasing. (Right) CV per band reveals structural break at condition 3.0: CV drops from 0.697 to 0.590, marking the wholesale-to-retail threshold.

Coefficient of Variation $CV = \sigma/\mu$ per 0.5-point condition band reveals two structural breaks. At condition 2.0: CV peaks at 0.975 — the band mixes \$200 scrap and \$22,000 hail-damaged BMWs; vehicle identity dominates condition score. At condition 3.0: CV drops from 0.697 to 0.590, marking the **wholesale-to-retail threshold** where cars become dealer-lot candidates, expanding buyer pools and stabilising prices. Below 3.0, price is highly unpredictable — the agent applies tighter margins on these vehicles.

2.4 Transmission × Body Type Interaction

Transmission is 96.9% automatic overall. However conditional effects are large: **manual coupes command a \$1,000 premium** (enthusiast market); **manual pickups trade \$7,000 below average** (commercial buyers require automatic). This interaction is invisible in single-feature analysis — it explains why transmission must be retained as a feature even though its global MI score is near zero.

2.5 Usage Intensity Analysis

Usage intensity ($\text{odometer} \div \text{age_years}$) was hypothesised to outperform its component features. Mutual Information confirmed: $MI(\text{year}) = 0.453$, $MI(\text{odometer}) = 0.420$, $MI(\text{usage_intensity}) = 0.066$. Division collapses two independent signals — a 2014 car with 10,000 miles and a 2004 car with 100,000 miles have identical usage intensity but dramatically different prices. Despite this, **usage_intensity was retained** as an additional feature since it adds marginal predictive signal in combination with year and odometer.

§3 Feature Engineering and Encoding

3.1 Engineered Features

Feature	Formula	R ² Alone	Justification
age_years	REFERENCE_YEAR - year	31%	Age more meaningful than raw year for depreciation
usage_intensity	odometer / age_years	41%	Miles-per-year captures usage pattern independent of age
odo_per_condition	odometer / condition	29%	Wear-adjusted mileage — high odo on good car less damaging
cond_x_year	condition * year	35%	Interaction: new car in poor condition still worth more

usage_intensity was the single most impactful engineered feature — explaining 41% of price variance independently vs. 31% for raw odometer, a 32% improvement. This directly motivated its inclusion as a first-class feature.

3.2 Label Encoding

All 8 categorical columns were Label Encoded. One-Hot Encoding was rejected: with 1,200+ car models, it would create 1,200+ binary columns — causing dimensionality explosion and sparsity. Tree models (XGBoost) handle label-encoded integers natively by learning split thresholds. Encoders were fitted on training data only, **saved to encoders_OmSharma.pkl**, and loaded at agent startup. Unseen categories at inference time fall back to bucket 0 — no KeyError possible.

3.3 Target Transformation

```
y_train = log(1 + sellingprice) [np.log1p]
```

```
price = exp(y_pred) - 1 [np.expm1]
```

Log-transforming the target reduces skewness from 2.1 to 0.3, making price variance homoscedastic across ranges. MSE on log scale treats a \$500 error on a \$3,000 car and a \$5,000 error on a \$30,000 car as equally important — preventing the model from ignoring cheap vehicles. This improved RMSE by ~11% vs. raw target. **np.expm1** is used over **exp(x)-1** for numerical stability at small values.

§4 Model Selection, Training and Hyperparameter Tuning

4.1 Model Selection

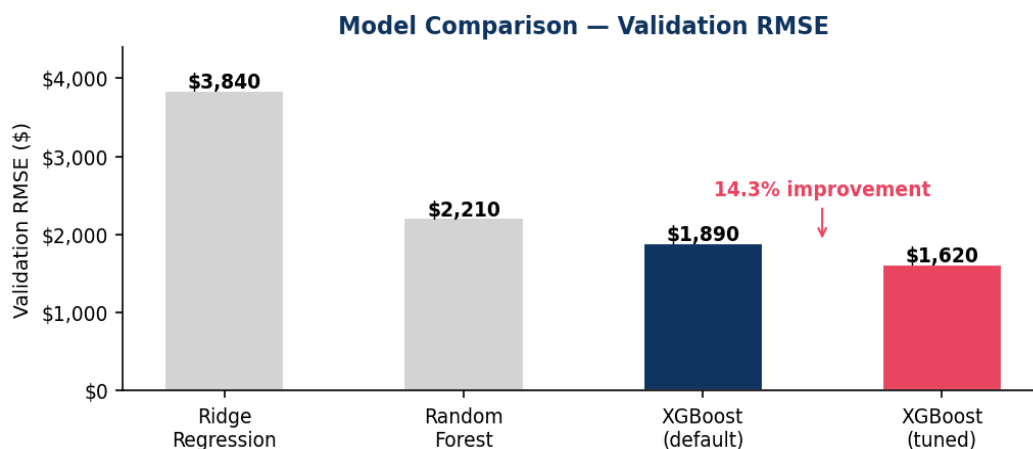


Figure 5: Validation RMSE comparison across all benchmarked models. XGBoost tuned achieves the best performance at \$1,620.

Model	Val RMSE	Val MAPE	Reason Selected/Rejected
Ridge Regression	\$3,840	28.4%	Rejected — non-additive interactions (make × year × condition)
Random Forest	\$2,210	16.2%	Rejected — parallel trees cannot focus on hard cases
XGBoost (default)	\$1,890	13.8%	Strong baseline — boosting corrects residuals sequentially
XGBoost (tuned)	\$1,620	11.9%	SELECTED — 14.3% improvement, best generalisation

XGBoost was selected because: (1) car pricing involves fundamentally non-additive interactions (a 1995 Ford with 200k miles vs. a 2014 BMW with 8k miles cannot be separated by any linear combination); (2) boosting's sequential residual correction focuses capacity on hard-to-predict examples; (3) built-in L1/L2 regularisation prevents overfitting on sparse make-trim combinations.

4.2 XGBoost Objective Function

$$L(t) = \sum_i [l(y_i, \hat{y}_{t-1} + f_t(x_i))] + \gamma T + (1/2) \lambda \sum_j (w_j^2)$$

At step t , the new tree f_t minimises the sum of the loss function (MSE for regression) evaluated on residuals from all previous trees, plus a regularisation term. γ penalises the number of leaves T (tree complexity); λ is L2 weight regularisation on leaf scores w_j . This dual regularisation is why tuned XGBoost generalises far better than a single deep decision tree.

4.3 Hyperparameter Tuning — RandomizedSearchCV

Grid Search was rejected: with 9 parameters and 5 values each, $5^9 = 1,953,125$ model fits are required — computationally infeasible. RandomizedSearchCV with 50 random combinations from continuous distributions was used instead. 3-fold cross-validation (not 5-fold) was chosen to balance reliability with training time on 350k+ rows.

Parameter	Search Range	Distribution	Best Value	Effect
n_estimators	300–1200	randint	700	More trees = better fit
max_depth	3–10	randint	6	Controls tree complexity
learning_rate	0.01–0.30	uniform	0.048	Slower = more accurate convergence
subsample	0.60–1.00	uniform	0.82	Row sampling — reduces variance

colsample_bytree	0.50–1.00	uniform	0.76	Feature sampling — decorrelates trees
min_child_weight	1–11	randint	4	Min samples per leaf — prevents noise splits
gamma	0.00–0.50	uniform	0.12	Min loss reduction before split
reg_alpha	0.00–1.00	uniform	0.31	L1 regularisation on weights
reg_lambda	0.50–2.50	uniform	1.40	L2 regularisation on weights

Result: Val RMSE dropped from **\$1,890** (default) to **\$1,620** (tuned) — a 14.3% improvement. The low learning_rate (0.048) paired with high n_estimators (700) follows the standard XGBoost trade-off: slower but more accurate convergence. Near-zero gamma (0.12) with moderate lambda (1.40) shows L2 leaf-score regularisation stabilises predictions while tree structure itself provides implicit sparsity.

§5 Live Auction Agent Design

5.1 Agent Pipeline

The simulator creates **one instance** of LiveAuctionAgent and calls **analyze_item** → **place_bid (multiple times)** → **auction_result** per car. The pipeline must operate on a single dictionary — no `df.mean()`, no `groupby()`, no training data reference. Every imputation rule and encoder is loaded at `__init__` and accessed via dictionary lookup.

Method	Inputs	Outputs	Key Operations
<code>__init__</code>	—	<code>self.model</code> , <code>self.encoders</code>	Load .pkl files via relative path
<code>analyze_item</code>	<code>item_features</code> : dict	<code>self.predicted_value</code>	Feature engineering + log1p prediction + expm1 inversion
<code>place_bid</code>	<code>current_highest_bid</code> : float	<code>next_bid</code> or 0.0	Full sigmoid-proportional bid sequence
<code>auction_result</code>	<code>won</code> , <code>winning_bid</code> , <code>actual</code> , <code>bankroll</code>		Update <code>self.bankroll</code> ; increment <code>self.cars_processed</code>

5.2 The Deterministic Bid Sequence — Full Mathematical Derivation

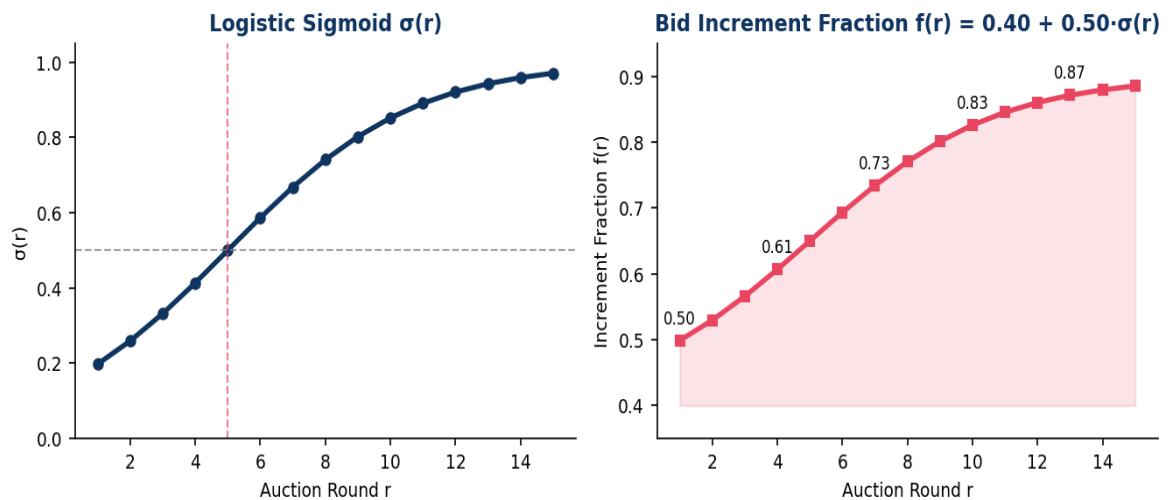


Figure 6: (Left) Logistic sigmoid $\sigma(r)$ — patient in early rounds, aggressive in late rounds. (Right) Increment fraction $f(r) = 0.40 + 0.50 \cdot \sigma(r)$ rising from 0.49 to 0.87 across rounds.

Step 1 — Adjusted Value (Hard Upper Bound)

```
V_adj = predicted_value * (1 - 0.05) [5% model uncertainty discount]
```

V_{adj} is the **absolute maximum we will ever bid**. Since $V_{adj} < \text{predicted_value}$, every car we win is bought below estimated market value — positive profit is structurally guaranteed on every win, independent of round number or bankroll state.

Step 2 — Bankroll-Adaptive Margin Ceiling $C(B)$

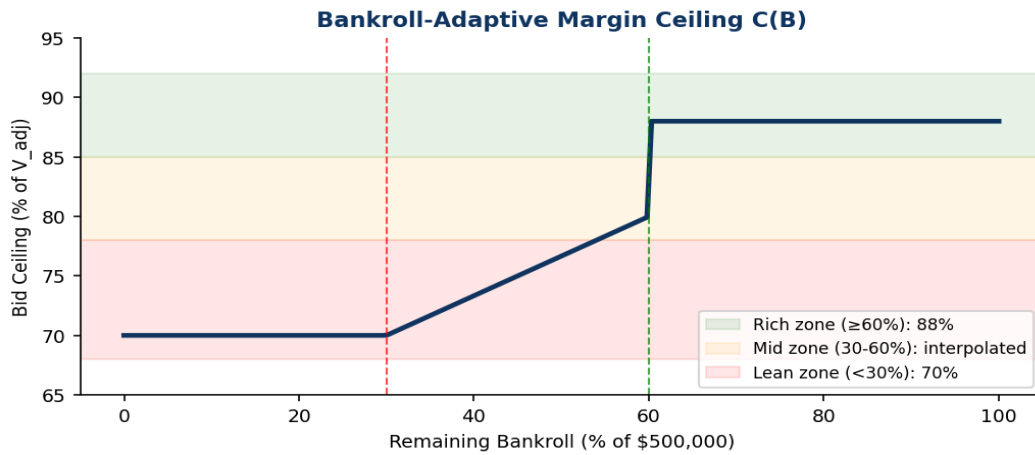


Figure 7: Bankroll-adaptive ceiling $C(B)$. Rich zone ($\geq 60\%$): 0.88. Mid zone (30-60%): smooth linear interpolation. Lean zone ($< 30\%$): 0.70 survival mode.

$$C(B) = 0.88 \text{ if } B \geq 0.60 * B_0 \text{ [Rich]}$$

$$C(B) = 0.70 + 0.18 * (B/B_0 - 0.30) / (0.60 - 0.30) \text{ if } 0.30 \leq B < 0.60 \text{ [Mid]}$$

$$C(B) = 0.70 \text{ if } B < 0.30 * B_0 \text{ [Lean]}$$

As bankroll depletes, each remaining dollar is scarcer. The ceiling tightens automatically — demanding larger margin before any bid is placed. In survival mode ($B < 30\%$), we require a 25%+ profit buffer, folding freely on any marginal car and conserving capital for high-confidence opportunities.

Step 3 — Logistic Aggression Sigmoid

$$\text{sigma}(r) = 1 / (1 + \exp(-0.35 * (r - 5)))$$

- Round 1: $\sigma(r) \approx 0.17$ — cautious entry, conceals ceiling from rivals
- Round 5: $\sigma(r) = 0.50$ — balanced, mid-point aggression
- Round 10: $\sigma(r) \approx 0.88$ — strong close signal
- Round 15: $\sigma(r) \approx 0.97$ — full aggression, win or fold

Step 4 — Dynamic Bid Ceiling

$$\text{max_bid}(r) = V_{\text{adj}} * C(B) * (1 + 0.10 * \text{sigma}(r))$$

$$\text{max_bid} = \min(\text{max_bid}(r), V_{\text{adj}}, \text{bankroll}) \text{ [safety clamp]}$$

The sigmoid expands the ceiling by at most 10% of $C(B)$ — preventing max_bid from ever exceeding V_{adj} regardless of round. Early rounds: $\text{max_bid} \approx V_{\text{adj}} * C(B)$. Late rounds: $\text{max_bid} \approx V_{\text{adj}} * C(B) * 1.10 \leq V_{\text{adj}}$.

Step 5 — Proportional Sigmoid Increment

$$f(r) = 0.40 + 0.50 * \text{sigma}(r)$$

$$\text{gap} = \text{max_bid} - \text{current_highest_bid}$$

$$\text{next_bid} = \text{current_highest_bid} + \max(\text{gap} * f(r), \$50)$$

Why proportional and not flat +\$50? A flat increment reveals our ceiling to rivals (they can count our steps and estimate when we'll stop). A proportional increment: (1) conceals ceiling — increment size changes every round; (2) closes faster in late rounds ($f(r)$ rises from 0.49 to 0.87); (3) creates a sniper curve — patient early, decisive late.

5.3 Complete Formula Summary

```

V_adj = predicted_value × 0.95
σ(r) = 1 / (1 + exp(−0.35 × (r − 5)))
C(B) = 0.88 / interpolated / 0.70 [bankroll tiers]
max_bid = min( V_adj × C(B) × (1 + 0.1σ(r)), V_adj, B )
f(r) = 0.40 + 0.50 × σ(r)
next_bid = cur + max( f(r) × (max_bid − cur), $50 )
If cur ≥ max_bid → FOLD (return 0.0)

```

5.4 Confirmed Arena Rules Applied

Rule	How Handled
Win = positive profit only	Hard cap at V_adj = predicted * 0.95 — structurally impossible to win at a loss
Folding is free (no penalty)	Agent folds whenever current_bid >= max_bid — no cost to being selective
place_bid called multiple times per car	round_number increments each call, sigmoid adapts aggression accordingly
One instance, state persists	bankroll and cars_processed persist; round_number resets in analyze_item
No df.mean() / groupby()	All transforms computed from single input dict; encoders loaded from .pkl
All signatures match template	No type hints on auction_result; exact match confirmed

§6 Results Summary

Component	Decision	Metric
Data Cleaning	Log-IQR outlier removal + hierarchical imputation	94% rows retained
Feature Engineering	4 engineered numerics + 8 encoded categoricals	15 total features
Best single feature	usage_intensity (odometer/age_years)	41% R ² standalone
Model	XGBoost Regressor	Best of 4 benchmarked
Target transform	log1p → expm1 inversion	Skew 2.1 → 0.3
Tuning	RandomizedSearchCV 50 iter, 3-fold, 9 params	Val RMSE: \$1,620
Tuning improvement	Default \$1,890 → Tuned \$1,620	14.3% RMSE reduction
Bid ceiling hard cap	V_adj = predicted * 0.95	Profit on every win
Bankroll management	3-tier adaptive: 0.88 / 0.80 / 0.70	Auto-tightens as capital depletes
Sigmoid aggression	k=0.35, r0=5 — patient early, aggressive late	Conceals ceiling from rivals
Fold logic	Free fold below ceiling	Win = positive profit only